MACHINE LEARNING LABORATORY
Experiment 07
Decision Tree Classification using ID3

Aim

To implement a Decision Tree classification algorithm using the ID3 approach and Information Gain, and use the resulting tree to classify new observations.

1. Objectives

After completing this experiment, students should be able to:

1. Implement the ID3 Decision Tree algorithm using Python.
2. Calculate Entropy and Information Gain programmatically.
3. Select the best attribute for splitting the dataset.
4. Recursively construct a Decision Tree.
5. Use the constructed tree to classify a new observation.
6. Evaluate the predictions using classification accuracy.
7. Understand how the implementation corresponds to the ID3 algorithm studied in theory.

2. Dataset

The **Play Tennis dataset** is used in this experiment.

The dataset contains categorical weather-related attributes and a binary target variable `Play`.

Outlook	Temperature	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

There are:

14 observations

and

4 input attributes

with:

Target = `Play`

where:

`Yes` = Play Tennis

`No` = Do Not Play Tennis

3. ID3 Implementation Flow

The implementation follows the ID3 procedure discussed in the theory class.

ID3 Algorithm Flow

1. Start with the complete training dataset.
2. Calculate the entropy of the current dataset.
3. Calculate Information Gain for every available attribute.
4. Select the attribute having the highest Information Gain.
5. Make that attribute the decision node.
6. Divide the dataset according to the values of that attribute.
7. Repeat the process recursively for every subset.
8. Stop when:
   - all observations belong to the same class, or
   - no attributes remain.
9. Assign the appropriate class to the leaf node.

4. Implementation

Step 1: Import Libraries

```
import pandas as pd
import math
```

Here, `pandas` is used to manipulate the dataset and `math` is used for logarithmic calculations.

Step 2: Create the Dataset

```
data = {
"Outlook": [
"Sunny", "Sunny", "Overcast", "Rain",
"Rain", "Rain", "Overcast", "Sunny",
"Sunny", "Rain", "Sunny", "Overcast",
"Overcast", "Rain"
],

'''
"Temperature": [
    "Hot", "Hot", "Hot", "Mild",
    "Cool", "Cool", "Cool", "Mild",
    "Cool", "Mild", "Mild", "Mild",
    "Hot", "Mild"
],
```

```
"Humidity": [
    "High", "High", "High", "High",
    "Normal", "Normal", "Normal", "High",
    "Normal", "Normal", "Normal", "High",
    "Normal", "High"
],

"Wind": [
    "Weak", "Strong", "Weak", "Weak",
    "Weak", "Strong", "Strong", "Weak",
    "Weak", "Weak", "Strong", "Strong",
    "Weak", "Strong"
],

"Play": [
    "No", "No", "Yes", "Yes",
    "Yes", "No", "Yes", "No",
    "Yes", "Yes", "Yes", "Yes",
    "Yes", "No"
]
'''

}

df = pd.DataFrame(data)

print(df)
```

Step 3: Calculate Entropy

The entropy function calculates the impurity of the target variable.

```
def entropy(target):

'''
values = target.value_counts()
total = len(target)

entropy_value = 0

for count in values:
    probability = count / total
    entropy_value -= probability * math.log2(probability)

return entropy_value
'''
```

Test the function:

```
print("Entropy of dataset:",
entropy(df["Play"]))
```

For the complete dataset, the target contains:

$$9\ Yes$$

and

$$5\ No$$

Therefore:

$$Entropy(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14}$$

which is approximately:

$$Entropy(S) \approx 0.940$$

Step 4: Calculate Information Gain

Information Gain is calculated for each attribute.

```
def information_gain(data, attribute, target_name):

'''
total_entropy = entropy(data[target_name])

values = data[attribute].unique()

weighted_entropy = 0

for value in values:

    subset = data[data[attribute] == value]

    probability = len(subset) / len(data)

    weighted_entropy += (
        probability *
        entropy(subset[target_name])
    )

gain = total_entropy - weighted_entropy

return gain
'''
```

Step 5: Calculate Information Gain for All Attributes

```
attributes = [
"Outlook",
"Temperature",
"Humidity",
```

```
"Wind"
]

for attribute in attributes:

'''
gain = information_gain(
    df,
    attribute,
    "Play"
)

print(attribute, ":", gain)
'''
```

Students should observe that the Information Gain values are different for different attributes.

The attribute with the largest Information Gain is selected as the root node.

For this dataset, the first split is:

Outlook

Step 6: Find the Best Attribute

Instead of manually selecting the attribute, create a function that automatically selects the attribute with maximum Information Gain.

```
def best_attribute(data, attributes, target_name):

'''
best_gain = -1
best_attr = None

for attribute in attributes:

    gain = information_gain(
        data,
        attribute,
        target_name
    )

    if gain > best_gain:
        best_gain = gain
        best_attr = attribute

return best_attr
'''
```

Test the function:

```
attributes = [
"Outlook",
```

```
"Temperature",
"Humidity",
"Wind"
]

print(
"Best attribute:",
best_attribute(df, attributes, "Play")
)
```

Expected output:

```
Best attribute: Outlook
```

Step 7: Implement the ID3 Algorithm Recursively

The following function implements the main ID3 algorithm.

```
def id3(data, attributes, target_name):

'''
# Case 1: All samples belong to one class
if len(data[target_name].unique()) == 1:
    return data[target_name].iloc[0]

# Case 2: No attributes remain
if len(attributes) == 0:
    return data[target_name].mode()[0]

# Select the best attribute
best_attr = best_attribute(
    data,
    attributes,
    target_name
)

tree = {best_attr: {}}

# Create a branch for every value
for value in data[best_attr].unique():

    subset = data[
        data[best_attr] == value
    ]

    remaining_attributes = [
        attr for attr in attributes
        if attr != best_attr
    ]

    tree[best_attr][value] = id3(
        subset,
```

```
        remaining_attributes,
        target_name
    )

return tree
'''
```

Step 8: Build the Decision Tree

```
attributes = [
"Outlook",
"Temperature",
"Humidity",
"Wind"
]

tree = id3(
df,
attributes,
"Play"
)

print(tree)
```

The output represents the learned Decision Tree as a nested Python dictionary.
A simplified representation of the learned tree is:

Outlook

Sunny *Overcast* *Rain*
↓ ↓ ↓

Further decisions are made for the branches that are not already pure.

5. Prediction using the Constructed Tree

Once the tree has been constructed, it can be used to classify a new observation.
For example:

Attribute	Value
Outlook	Sunny
Temperature	Cool
Humidity	High
Wind	Strong

The prediction process follows the tree from the root node until a leaf is reached.

Step 9: Prediction Function

```
def predict(tree, sample):

'''
if not isinstance(tree, dict):
    return tree

attribute = list(tree.keys())[0]

value = sample[attribute]

subtree = tree[attribute][value]

return predict(subtree, sample)
'''
```

Step 10: Predict a New Sample

```
new_sample = {
"Outlook": "Sunny",
"Temperature": "Cool",
"Humidity": "High",
"Wind": "Strong"
}

prediction = predict(tree, new_sample)

print("Prediction:", prediction)
```

The output will be either:

```
Prediction: Yes
```

or

```
Prediction: No
```

depending on the learned tree.

6. Testing the Model on the Dataset

Students can also use the constructed tree to generate predictions for the existing observations.

```
predictions = []

for index, row in df.iterrows():

'''
sample = row.drop("Play").to_dict()
```

```
prediction = predict(tree, sample)

predictions.append(prediction)
'''

df["Predicted"] = predictions

print(df)
```

Accuracy

Calculate the number of correctly classified observations.

```
correct = sum(
df["Play"] == df["Predicted"]
)

accuracy = correct / len(df)

print("Accuracy:", accuracy)
```

The accuracy is:

$$Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$$

Important Observation

In this part of the experiment, students are implementing the Decision Tree logic themselves rather than simply calling a ready-made Decision Tree classifier.

7. Verify using Scikit-learn

After implementing ID3 manually, students can verify the concept using Scikit-learn.

```
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Since the dataset contains categorical values, encode them numerically.

```
df_encoded = df.drop("Predicted", axis=1).copy()

encoders = {}

for column in df_encoded.columns:

'''
encoder = LabelEncoder()

df_encoded[column] = encoder.fit_transform(
```

```
    df_encoded[column]
)

encoders[column] = encoder
```

Separate input and target:

```
X = df_encoded.drop("Play", axis=1)

y = df_encoded["Play"]
```

Create the Decision Tree:

```
model = DecisionTreeClassifier(
criterion="entropy",
random_state=42
)

model.fit(X, y)
```

Predict:

```
y_pred = model.predict(X)

accuracy = accuracy_score(y, y_pred)

print("Accuracy:", accuracy)
```

Here:

$$\texttt{criterion="entropy"}$$

is used because the theoretical algorithm studied in this experiment is based on:

$$\boxed{\text{ID3} \rightarrow \text{Entropy} + \text{Information Gain}}$$

8. Visualization using Scikit-learn

The learned tree can also be visualized.

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(16, 9))

plot_tree(
model,
feature_names=X.columns,
class_names=["No", "Yes"],
filled=True
)

plt.show()
```

Students should identify:

- Root node
- Internal decision nodes
- Branches
- Leaf nodes
- Splitting conditions
- Predicted class

9. Observations

Complete the following table.

Observation	Result
Total number of observations	
Number of input attributes	
Target attribute	
Entropy of complete dataset	
Attribute selected as root	
Highest Information Gain	
Manual ID3 accuracy	
Scikit-learn accuracy	

10. Result

The ID3 Decision Tree classification algorithm was implemented using Python.

Entropy and Information Gain were calculated programmatically, the best splitting attribute was selected recursively, and a Decision Tree was constructed.

The constructed tree was used to classify observations and its performance was evaluated using classification accuracy.

11. Viva Questions

1. What is a Decision Tree?
   “‘
2. What is the difference between a root node and a leaf node?
3. What is the purpose of Entropy in ID3?

4. What does Information Gain represent?
5. Why is the attribute with the highest Information Gain selected?
6. What is the stopping condition in ID3?
7. Why is ID3 called a recursive algorithm?
8. Why is feature scaling generally not required for Decision Trees?
9. What is the difference between ID3 and a Decision Tree implementation using Gini impurity?
10. What problem can occur if a Decision Tree becomes very deep?
11. What is overfitting?
12. Why do we keep a separate test dataset when evaluating a machine learning model?

‟‟

12. Student Tasks

1. Execute the complete ID3 implementation.

   ‟‟

2. Calculate and display the Information Gain of:
   - Outlook
   - Temperature
   - Humidity
   - Wind
3. Identify the attribute selected as the root node.
4. Display the constructed Decision Tree.
5. Predict the class of the following observation:

Attribute	**Value**
Outlook	Rain
Temperature	Cool
Humidity	Normal
Wind	Strong

6. Create at least three additional test observations and predict their classes.
7. Calculate the classification accuracy.
8. Verify the Decision Tree using Scikit-learn with `criterion="entropy"`.
9. Compare the manually implemented tree with the Scikit-learn tree. ‟‟

13. Important Learning Point

From Theory to Implementation

In the theory class, the ID3 process was studied as:

$$\boxed{\text{Entropy} \rightarrow \text{Information Gain} \rightarrow \text{Best Attribute} \rightarrow \text{Split} \rightarrow \text{Recursion}}$$

In this experiment, the same process is converted into Python functions.

$$\boxed{\text{Theory Algorithm} \longrightarrow \text{Python Implementation} \longrightarrow \text{Decision Tree} \longrightarrow \text{Prediction}}$$